

Introduction

Zenlang is a modern, minimal, optionally typed programming language designed for clarity, flexibility, and rapid development. It blends ideas from Python, JavaScript, and more traditionally typed languages, while maintaining a unique architecture for scripting, application development, and compiled execution.

This document introduces the language philosophy, goals, and guiding principles.

Zenlang is a high-level programming language designed to be:

- Beginner-friendly with readable syntax inspired by Python & JavaScript
- Optionally statically typed (types can be declared or inferred)
- Object-oriented, with classes and mutable objects
- Capable of both interpreted and compiled execution
- Async/await capable to support concurrent operations
- Suitable for CLI, GUI, and web applications
- Supports ZenScript (.zs) for quick, dynamic execution
- Supports ZenLang (.zl) for compiled, optimized binaries

The language focuses on simplicity, extensibility, and ease of prototyping while remaining powerful for full-scale applications.

Source Code Representation

Zenlang source files are UTF-8 encoded text. Whitespace generally has no semantic meaning except to separate tokens. Newlines may influence indentation in the future but currently do not affect parsing.

Whitespace is ignored except inside:

- Strings
- Multiline strings
- Comments

Indentation is not syntactically significant.

Comments

Comments do not nest.

Single Line Comment

Syntax:

```
// ...
```

Example:

```
// this is a single line comment
```

Multi-line Comment

Syntax:

```
/* ... */
```

Example:

```
/*  
  This  
  is  
  a  
  multi-line  
  comment  
*/
```

Documentation Comment

Syntax:

```
/*! ... !/
```

Example:

```
/*!  
  This  
  is  
  a  
  multi-line  
  documentation  
  comment  
*/
```

Identifiers

Identifiers are names used to identify variables, functions, classes, types, and other symbols.

An identifier consists of:

- a leading letter or underscore
- followed by letters, digits, or underscores

Identifiers are case-sensitive.

Identifiers

Syntax:

```
[a-zA-Z_][a-zA-Z0-9_]*
```

Example:

```
UPPERCASE  
lowercase  
camelCase  
Snake_Case  
PascalCase
```

Scope Blocks

Zen supports:

- Global scope
- Lexical scope (single and nested)
- Named scope (explicit, addressable)

Scope Blocks use:

Single Scope

Syntax:

```
{ ... }
```

Nested Scope

Syntax:

```
{ ...  
  { ... }  
}
```

Named Scope

Syntax:

```
scope Identifier { ... }  
  
Identifier.Identifier
```

Scopes

Example:

```
/* Scope Level 0 : Global Scope */

{ /* Scope Level 1 */ }

{ /* Scope Level 1 */
  { /* Scope Level 2 */ }
}

scope NamedScope { /* Scope Level 1 : NamedScope */ }
```

Single, Nested and Named Scope Blocks:

```
/* Scope Level 0 : Global Scope */

{ /* Scope Level 1 */ }

{ /* Scope Level 1 */
  { /* Scope Level 2 */ }
}

scope NamedScope { /* Scope Level 1 : Named Scope */ }
```

Global Scope

Exactly one global scope per program/module.

Contains:

- Builtins
- Global constants and variables
- Global functions, structures, enumerators
- Global named scopes

Rules:

- Always visible
- Cannot reference child scopes
- Exists for the lifetime of the program

Example:

```
set VERSION : String -> `0.1.0`

function main {
  write(VERSION)
}
```

Lexical Scopes (single and nested)

These are implicit scopes, created by syntax. Scopes can nest arbitrarily.

Single scopes:

- One parent
- One lifetime
- No name
- No external visibility

Example:

```
{  
  let x -> 10  
}
```

Nested scopes:

- Inner scopes can read parent symbols
- Parents cannot read child symbols
- Shadowing allowed

Example:

```
{  
  let x -> 10  
  {  
    let y -> x + 5  
  }  
}
```

Named Scopes (explicit scopes)

Think of it as a labeled block.

Named scope:

- A scope with an identifier
- Lexically nested
- Addressable by name
- Not a namespace (important distinction)

Example:

```
scope Config {  
  set MAX_HP : Integer -> 100  
}  
  
write(Config.MAX_HP)
```

Use cases:

```
scope GameConfig {  
  set MAX_PLAYERS : Integer -> 64  
  set TICK_RATE : Integer -> 60  
}
```

```
scope PlayerState {  
  let health -> 100  
  let stamina -> 50  
}
```

```
scope Platform {  
  
  scope Windows {  
    set PATH_SEPERATOR : String -> `\  
  }  
  
  scope Linux {  
    set PATH_SEP : String -> `/  
  }  
  
}  
  
write(Platform.Linux.PATH_SEP)
```

Types

Zenlang is optionally typed: You may declare types, or let them be inferred.

Zenlang defines several Types:

Primitive Types

Void

No returned value.

Syntax:

```
Void
```

Example:

```
function test : Void { write(`Hello, World!`) }
```

Variant

Stores any Zen Type and able to change value types freely.

Syntax:

```
Variant
```

Example:

```
function test : Variant { <- `Hello, World!` }
```

Integer

Defaults to a 32-bit signed integer.

Syntax:

```
Integer           // 32-bit signed integer default
Integer[Size]     // Size in bits

Integer[8]        // 8-bit signed integer
Integer[16]       // 16-bit signed integer
Integer[32]       // 32-bit signed integer
Integer[64]       // 64-bit signed integer
```

Example:

```
let a : Integer -> 10
function test : Integer { <- 10 }
```

Decimal

Defaults to a 32-bit floating point.

Syntax:

```
Decimal           // 32-bit floating point default
Decimal[Size]     // Size in bits

Decimal[32]       // 32-bit floating point
Decimal[64]       // 64-bit floating point
```

Example:

```
let a : Decimal -> 10.0
function test : Decimal { <- 10.0 }
```

Boolean

True or False.

Syntax:

```
Boolean
```

Example:

```
let a : Boolean -> True
function test : Boolean { <- True }
```


String

UTF-32 default

Syntax:

```
String          // UTF-32 default
String[Size]    // Encoding Size in bits

String[8]       // UTF-8   (8bits - 1 byte)
String[16]      // UTF-16  (16bits - 2 bytes)
String[32]      // UTF-32  (32bits - 4 bytes)
```

Example:

```
let a : String -> `Hello, World!`
function test : String { <- `Hello, World!` }
```

Example:

```
x``           // Hex encoded string

b16``         // Base16 encoded string
b32``         // Base32 encoded string
b64``         // Base64 encoded string

u``           // Unicode encoded string
u8``          // UTF-8 encoded string
u16``         // UTF-16 encoded string
u32``         // UTF-32 encoded string
```

Example:

```
f``           // Format string
f`{a} : {b}`  // Format string with arguments

t``           // Template string
t`{a} : {b}`  // Template string with arguments
```

Collection Types

Map

Stores Key Value pairs.

Syntax:

```
Map
Map<Type, Type>
```

Example:

```
let a : Map<String, Integer> -> { `a` -> 1, `b` -> 2 }
function test : Map { <- { `a` -> 1, `b` -> 2 } }
```

List

Stores a list of values in a fixed order.

Syntax:

```
List  
List<Type>
```

Example:

```
let a : List -> { 1, 2 }  
  
let a : List<Integer> -> { 1, 2, 3, 4, 5 }  
let a : List<String> -> { `a`, `b`, `c`, `d`, `e` }  
let a : List<Variant> -> { `a`, 1, True }  
  
function test : List<Integer> { <- { 1, 2, 3, 4, 5 } }
```

Set

Stores a list of unique values.

Syntax:

```
Set  
Set<Type>
```

Example:

```
let a : Set -> { 1, 2 }  
  
let a : Set<Integer> -> { 1, 2, 3, 4, 5 }  
let a : Set<String> -> { `a`, `b`, `c`, `d`, `e` }  
let a : Set<Variant> -> { `a`, 1, True }  
  
function test : Set<Integer> { <- { 1, 2, 3, 4, 5 } }
```

Tuple

Stores a list of Types in a fixed order. Tuple can be named.

Syntax:

```
Tuple  
Tuple<Type, ... >
```

Example:

```
let a : Tuple -> { 1, 2 }  
  
// Tuple with 2 elements of different types  
let a : Tuple<String, Integer> -> { `Attack`, 100 }  
  
// Tuple with 2 elements of the same type, but named  
function test : Tuple<Integer> { <- { `Attack` -> 100, `Defense` -> 100 } }
```

Vector

Stores a fixed length list of values in a fixed order

Syntax:

```
Vector  
Vector[Size]           // Size in elements  
Vector[Size]<Type>
```

Example:

```
let a : Vector[4] -> { 1, 2, 3, 4 }  
let a : Vector[4]<Integer> -> { 1, 2, 3, 4 }  
let a : Vector[4]<String> -> { `a`, `b`, `c`, `d` }  
let a : Vector[4]<Variant> -> { `a`, 1, True }  
  
function test : Vector[4]<Integer> { <- { 1, 2, 3, 4 } }
```

Container Types

```
Class           // Stores a list of [Function, Assignment] Types  
Object          // Stores a list of [Function, Assignment] Types  
Structure       // Stores a list of Structure Member Types  
Enumerator      // Stores a list of Enumerator Member Types
```

Action Types

```
Function        // Stores a list of Function Parameter Types  
Task            // Stores a list of [Function, Assignment] Types
```

Abstract Types

Number

Accepts Integer and Decimal values

Syntax:

```
Number
```

Example:

```
let a : Number -> 1
let a : Number -> 1.0

function add(a : Number, b : Number) : Number { <- a + b }
```

Text

Accepts String and Rune values

Syntax:

```
Text
```

Example:

```
let a : Text -> `Hello, World!`

function print(input : Text) : Void { write(input) }
```

```
Number      // Number Type (Integer, Decimal)
Text        // Text Type (String, Rune)
Container    // Container Type (Class, Object, Structure, Enumerator)
Collection   // Collection Type (Map, List, Set, Tuple, Vector)
```

User-Defined Types

Zenlang supports custom types via:

- classes
- structures
- enumerators

Classes support inheritance. Structures are simple value types. Enumerators define closed sets of named variants.

Enumerators

```
from zen.io import write

enumerator name { .... }

enumerator Direction { North, South, East, West }

write(Direction) // ( `North`, 0), (`South`, 1), (`East`, 2), (`West`, 3) )

enumerator Direction {
  North -> 1,
  South -> 2,
  East -> 3,
  West -> 4
}

write(Direction) // ( `North`, 1), (`South`, 2), (`East`, 3), (`West`, 4) )
write(Direction.names) // ( `North`, `South`, `East`, `West` )
write(Direction.values) // ( 1, 2, 3, 4 )
write(Direction.South) // (`South`, 2)
write(Direction.South.name) // `South`
write(Direction.South.value) // 2

enumerator Week {
  SUNDAY -> -1,
  MONDAY -> 1,
  TUESDAY -> 10,
  WEDNESDAY -> 100
}
write(Week) // ( `SUNDAY`, -1), (`MONDAY`, 1), (`TUESDAY`, 10), (`WEDNESDAY`, 100) )

enumerator Week {
  SUNDAY -> 10,
  MONDAY -> auto,
  TUESDAY -> auto,
  WEDNESDAY -> auto
}

write(Week) // ( `SUNDAY`, 10), (`MONDAY`, 11), (`TUESDAY`, 12), (`WEDNESDAY`, 13) )
```

Classes

```
from zen.io import write

class Person {
  let name : String -> `Unknown`
  let age : Integer -> 0

  function init(name : String, age : Integer) : Void {
    self.name -> name
    self.age -> age
    write(`Initialized ` + self.name)
  }

  function greet() : Void {
    write(`Hello, my name is ` + self.name + ` and I am ` + self.age + ` years old.`)
  }
}

let person : Person -> Person(`Alice`, 30)
write(person.greet())
```

```

from zen.io import write

class Animal {
  let name -> `Animal`

  function sound() {
    write(`Generic sound`)
  }
}

class Dog extends Animal {
  function sound() {
    write(`Woof!`)
  }
}

write(`-- Animal --`)
let animal -> Animal()
write(animal.name)
animal.sound()

write(`-- Dog --`)
let dog -> Dog()
write(dog.name)
dog.sound()

```

```

from zen.io import write

class Animal {
  function sound() {
    write(`Generic Animal Sound`)
  }
}

class Dog extends Animal {
  function sound() {
    write(`Dog says:`)
    parent.sound()
  }
}

let dog -> Dog()
dog.sound()

```

Abstract Types

Number

```

let x : Number -> 10
let y : Number -> 1.0
let z : Number -> 0x7B

```

Text

```

let x : Text -> `A`
let y : Text -> `This is a sentence`
let z : Text -> `Word`

```

Container

```
let x : Container -> function ( x ) <- x
let y : Container -> structure x , y
let z : Container -> enumerator A, B, C

let x : Container -> function ( x : Integer ): Integer {
  <- x
}
let y : Container -> structure {
  x -> 0.0
  y -> 1.0
}
let z : Container -> enumerator {
  A,
  B,
}
```

Collection

```
let x : Collection -> [1, 2, 3]
let y : Collection -> (`Hello`, 123)
```

Type Annotations

A variable or parameter may specify its type:

```
let x : Integer -> 10    // x : Integer
let y -> 10              // y : Variant
```

If no type is specified, Zenlang attempts to infer the type from context.

The special operator `:>` means “infer type from value” and may be used where explicit annotation syntax appears.

```
let x :> 10              // y : inferred Integer
```

Type Inference

Zenlang infers types in:

- variable declarations
- function returns
- arithmetic expressions
- list, tuple, and dictionary literals

Literals

Zenlang includes the following literal types:

Integer literals

Sequence of digits:

```
0
42
123456789
1_000_000
```

Floating-point literals

Contain a decimal point:

```
3.14159
0.5
.25
10.
```

Number literals

Number Type can hold (Integer, Decimal, Hexadecimal, Binary, Octal)

```
1          // Integer
0.5        // Decimal
0x7B       // Hexadecimal
0o173      // Octal
0b01111011 // Binary
1e-5       // Scientific

// Underscored Separators
1_000_000
0xDE_AD_BE_EF
0b1101_0001
```

Boolean literals

```
True
False
```

Nothing literal

```
Nothing
```

Rune and String literals

Runes and strings are both enclosed in backticks. A single-code-point backtick literal is a Rune and multiple-code-point backtick literal is a string.

```
`          // Empty string
`c`        // Rune
`hello world` // String
```

List literals

```
[]          // Empty list
[1, 2, 3, 4] // List of numbers
[1, `a`, True] // List of mixed types
```

Tuple literals

```
()          // Empty tuple
(1, 2, 3)    // Tuple of numbers
(1, `a`, True) // Tuple of mixed types
(`a` -> 1, `b` -> 2) // Named tuple
```

Dictionary literals

```
{ }          // Empty dictionary
{ `a` -> 1, `b` -> 2 } // Dictionary of Key: String and Value: Number
```

Generic literals

```
l{1, 2, 3}           // List
v{1, 2, 3}           // Vector
t{1, 2, 3}           // Tuple
d{'a' -> 1, 'b' -> 2} // Dictionary
```

Variables & Assignment

Variable (mutable)

Let Assignment Expression

Syntax:

```
let Identifier -> Expression
let Identifier : Type -> Expression
let Identifier :=> Expression
```

Example:

```
let a -> 10
let b : Integer -> 100
let c :=> `Hello World`
```

Constant (immutable)

Set Assignment Expression

Trying to reassign a constant raises an error.

Syntax:

```
set Identifier : Type -> Expression
set Identifier :=> Expression
```

Example:

```
set PI : Decimal -> 3.14159
set FPS :=> 60
```

Assignment typing

```
let x -> 1           // Type will be resolved to Variant and inferred as Integer
let y : Integer -> 2 // Type will be resolved as Integer
let z :=> 3           // Type will be resolved to Integer and inferred as Integer
```

Logic

```
True  
False
```

```
True and False  
True or False  
not True
```

Logical Operators

```
and    // Logical AND  
or     // Logical OR  
not    // Logical NOT  
xor    // Logical XOR  
nor    // Logical NOR  
nand   // Logical NAND  
xnor   // Logical XNOR
```

Bitwise Operators

```
&&     // Bitwise AND  
||     // Bitwise OR  
!!     // Bitwise NOT  
^^     // Bitwise XOR  
!|     // Bitwise NOR  
!&     // Bitwise NAND  
!^     // Bitwise XNOR  
%%     // Bitwise MOD  
<<     // Bitwise LEFT SHIFT  
>>     // Bitwise RIGHT SHIFT
```

Comparison Operators

```
=       // Equal  
!=      // NOT Equal  
&=     // AND Equal  
|=     // OR Equal  
^=     // XOR Equal  
%=     // MOD Equal  
<      // Less Than  
>      // Greater Than  
<=     // Less Than or Equal  
>=     // Greater Than or Equal  
<<=    // LEFT SHIFT Equal  
>>=    // RIGHT SHIFT Equal
```

Arithmetic Operators

```
+      // Addition
-      // Subtraction
*      // Multiplication
/      // Division
%      // Remainder (Modulo)
:      // Range
**     // Exponentiation (Power)
++     // Increment
--     // Decrement
//     // Quotient
()     // Parentheses
```

Order of Operations

```
1 + (2 * 2)
(2 + 2) * (8 / 2) + 10
(4 * 4 / (4 + 4))
```

Control Flow

Zenlang features several flexible control flow constructs.

Conditionals

When

Syntax:

```
when condition { .... }

when condition { .... } or { .... }

when condition { .... } or condition { .... } or { .... }
```

Example:

```
when x > 0 {
  write(`positive`)
} or {
  write(`negative`)
}
```

When Inline

Syntax:

```
value when condition or value
```

Example:

```
let message -> `positive` when x > 0 or `negative`
```

Defer

Defer runs when the enclosing function returns, even if it panics or returns early.

Defer

Syntax:

```
defer expression  
  
defer { .... }
```

Example:

```
function test {  
  defer write(`Defer`)  
  write(`Hello`)  
}  
  
function test {  
  let a -> 1  
  defer {  
    write(`Defer`)  
    a -> 100  
    write(a) // 100  
  }  
  write(a) // 1  
}
```

Loops

Do

Syntax:

```
do { .... }
```

Example:

```
do {  
  write(`Hello`)  
}
```

Do When

Syntax:

```
do when condition { .... }  
  
do when condition { .... } or { .... }
```

Example:

```
do when x > 0 {  
    write('Hello')  
}  
  
do when x > 0 {  
    write('Hello')  
} or {  
    write('World')  
}
```

Do While

Syntax:

```
do while condition { .... }  
  
do while condition { .... } or { .... }  
  
do { ... } while condition
```

Example:

```
// pre check  
do while condition {  
    ....  
}  
  
// or only with pre check  
do while condition {  
    ....  
} or {  
    ....  
}  
  
// post check  
do {  
    ....  
} while condition
```

Do Until

Syntax:

```
do until condition { .... }  
  
do { ... } until condition
```

Example:

```
// pre check  
do until condition {  
    ....  
}  
  
// post check  
do {  
    ....  
} until condition
```

Do For

Syntax:

```
do for value in container { .... }  
  
do for value in container { .... } or { .... }
```

Example:

```
// pre check  
do for value in container {  
    ....  
}  
  
// post check  
do for value in container {  
    ....  
} or {  
    ....  
}
```

Break / Continue

As expected in loops.

```
break  
continue
```

Function Declarations

Functions are introduced with the function keyword.

Functions Inline

Syntax:

```
function Identifier Expression  
  
function Identifier <- Expression  
  
function Identifier : ReturnType <- Expression  
  
function Identifier ( Parameters ) Expression  
  
function Identifier ( Parameters ) <- Expression  
  
function Identifier ( Parameters ) : ReturnType <- Expression
```

Example:

```
function hello_world write(`Hello World`)  
  
function hello_world <- `Hello World`  
  
function get_greeting : String <- `Hello World`  
  
function print : String ( message : String ) write(message)  
  
function add ( a, b ) <- a + b  
  
function sub : Integer ( a : Integer, b : Integer ) <- a - b
```

Anonymous Functions

Syntax:

```
let Identifier -> function { ... }

let Identifier : ReturnType -> function { ... }

let Identifier ( Parameters ) -> function { ... }
```

Example:

```
let hello_world -> function {
  write(`Hello World`)
}

let get_greeting : Function<String> -> function {
  <- `Hello World`
}

let print : Function<String> -> function ( message : String ) {
  write(message)
}

let add : Function<Integer> -> function ( a : Integer, b : Integer ) {
  <- a + b
}
```

Anonymous Functions Inline

Syntax:

```
let Identifier -> function ...

let Identifier -> function <- ...

let Identifier : ReturnType -> function <- ...

let Identifier ( Parameters ) -> function ...

let Identifier ( Parameters ) -> function <- ...
```

Example:

```
let hello_world -> function write(`Hello World`)

let get_greeting : Function<String> -> function <- `Hello World`

let print : Function<String> -> function ( message : String ) write(message)

let add : Function<Integer> -> function ( a : Integer, b : Integer ) <- a + b
```

Function Parameters

Syntax:

```
( Identifier, ... )  
  
( Identifier : Type, ... )  
  
( Identifier -> Value, ... )  
  
( Identifier : Type -> Value, ... )
```

Example:

```
( a, b )  
  
( a : Integer, b : Integer )  
  
( a -> 1, b -> 2 )  
  
( a : Integer -> 1, b : Integer -> 2 )
```

Function Variadic Parameters

Syntax:

```
( Identifier* )  
  
( Identifier** )
```

Example:

```
// Variadic Arguments  
function test ( args* ) {  
  write(args[0])  
  
  for arg in args {  
    write(arg)  
  }  
}  
  
test( 1, 2, 3 )  
  
// Variadic Keyword Arguments  
function test ( kwargs** ) {  
  write(kwargs.a)  
  
  for key, value in kwargs {  
    write(key)  
    write(value)  
  }  
}  
  
test( a -> 1, b -> 2, c -> 3 )  
  
test( a : Integer -> 1, b : Integer -> 2, c : Integer -> 3 )
```


Structure Inline

Syntax:

```
structure Identifier Identifier, ...  
  
structure Identifier Identifier -> Value, ...  
  
structure Identifier Identifier : Type, ...  
  
structure Identifier Identifier : Type -> Value, ...
```

Example:

```
structure Point x, y  
  
structure Point x -> 0, y -> 0  
  
structure Point x : Integer, y : Integer  
  
structure Point x : Integer -> 0, y : Integer -> 0
```

Structure Instantiation & Unpacking

Syntax:

```
let Identifier -> Identifier()  
  
let Identifier : Structure -> Identifier ( parameters )  
  
Identifier.Member
```

Example:

```
let point -> Point()  
  
let point : Point -> Point ( 0, 0 )  
  
let point : Point -> Point ( x -> 0, y -> 0 )  
  
point.x -> 10  
point.y -> 100  
  
write( point.x )  
write( point.sum() )  
  
let x, y -> point  
  
write(x)  
write(y)
```

Objects

Object

Syntax:

```
object Identifier { ... }

object Identifier { Identifier -> Value, ... }

object Identifier { Identifier : Type -> Value, ... }
```

Example:

```
object test {
  attack -> 100,
  defence -> 50
}

object test {
  attack : Integer -> 100,
  defence : Integer -> 50
}
```

Class Declaration

Classes define reference types with methods and fields.

```
class Name {
  let name : Type

  function name { .... }
}
```

Class Initialization

```
class Name {
  let name : Type -> expression

  function init ( parameters ) {
    self.name -> parameter
    statements
  }
}
```

Class Inheritance

```
class SuperClass {
  let name : Type -> expression

  function init ( parameters ) {
    self.name -> parameter when parameter or default
  }
}

class SubClass : SuperClass {
  function init ( parameters ) {
    parent.init(parameters)
    statements
  }
}
```

Class Instantiation

```
let name -> Class()
let name : Class -> Class ( parameters )

name.member
name.member -> value

name.member()
name.member ( parameters )
```

Modules

Zenlang supports importing and exporting code between files.

Importing

```
import math
import utils.helpers
from gui import Button

from zen.io import write
from zen.io import read as r
```

Exporting

```
export function foo { .... }
export class Person { .... }
```

Error Handling

Zenlang uses check, assert and raise to manage exceptions.

Check

```
check {
  ....
} or {
  write(error)
}
```

```
function calculate() {
  let result -> check divide(10, 0) or {
    write(`Something went wrong!`)
    <- 0 // Default value
  }

  write(`Result is: ` + result)
}
```

```
let file -> open(`config.zl`)

check file {
  case is Error {
    write(`Failed to open file`)
  },
  case is String {
    write(`File content: ` + file)
  }
}
```

```
check condition {
  cases
} or { default }           // check conditions

check condition or default  // check with default
? condition or default     // inline check with check symbol '?'

check condition raise error // check with raise
? condition ^ error        // inline check symbol '?' with raise symbol '^'
```

Raise

```
raise `ErrorUnknown`           // raise keyword

^ `ErrorUnknown`               // raise symbol '^'

raise `ErrorUnknown` with `Value` // raise with value
```

```
function divide(a: Decimal, b: Decimal) Decimal {
  when b == 0 {
    raise `DivisionByZeroError` // This immediately returns the error value
  }
  <- a / b
}
```

Assert

```
assert condition           // assert keyword
assert condition raise error // assert keyword with raise keyword

! condition                // assert symbol '!'
! condition ^ error        // assert symbol '!' with raise symbol '^'
```

- Symbol: ^, Keyword: raise, Purpose: Immediate return of an error value/variant.
- Symbol: ?, Keyword: check, Purpose: Unwrapping a value or branching based on a condition.
- Symbol: !, Keyword: assert, Purpose: Validation of assumptions; halts or raises on failure.

Semantic Comparison Table

Recovery: let x -> ? func() or 0 Propagation: ? condition ^ Error(ErrorName) Validation: ! x > 0 Matching: check value { case ... }

Error ReturnType

```
// Multiple Return Types '[Decimal, Error]'  
// Will return error if raise is called else return Decimal  
// Base ReturnType should be either Variant or TypeUnion?  
  
function divide : [Decimal, Error] (a: Decimal, b: Decimal) {  
  when b == 0 {  
    raise `DivisionByZeroError` // This immediately returns the error value  
  }  
  <- a / b  
}
```